PRACTICAL - 13

Aim: Generic containers (Box.java + Driver.java)

Code Screenshot:

Visual Studio Code - Box.java

Box.java

```java
public class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}

class Pair<A, B> {
    private final A first;
    private final B second;
    public Pair(A first, B second) { this.first = first; this.second = second; }
    public String toString() { return "(" + first + ", " + second + ")"; }
}
```

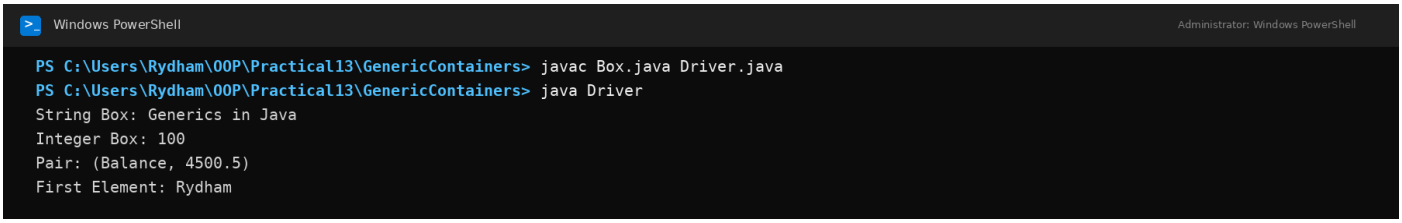
Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static <T> T getFirst(T[] values) {
        return values == null || values.length == 0 ? null : values[0];
    }

    public static void main(String[] args) {
        Box<String> text = new Box<>();
        text.set("Generics in Java");
        Box<Integer> number = new Box<>();
        number.set(100);
        Pair<String, Double> pair = new Pair<>("Balance", 4500.50);
        String[] names = {"Rydham", "Karan", "Rohan"};
        System.out.println("String Box: " + text.get());
        System.out.println("Integer Box: " + number.get());
        System.out.println("Pair: " + pair);
        System.out.println("First Element: " + getFirst(names));
    }
}
```

Output Screenshot:



Learning Outcome:

- Designed custom type-parameterized generic classes (Box).
- Handled multiple independent type variables (Pair).
- Implemented generic methods with independent type parameters (T getFirst).
- Enforced compile-time type safety without runtime casting.
- Understood how generics eliminate ClassCastException bugs.

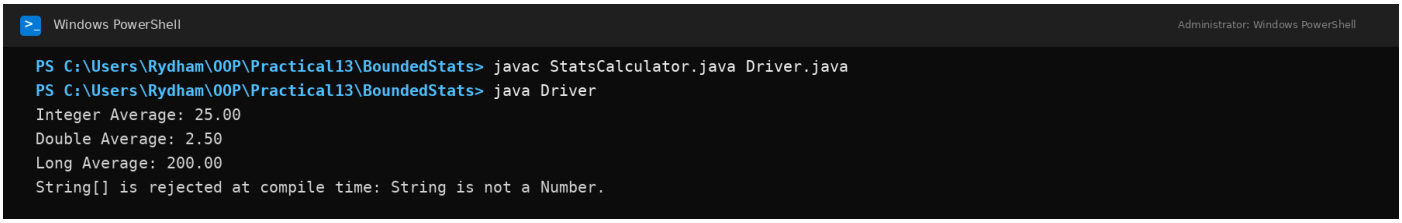
PRACTICAL - 13

Aim: Bounded stats (StatsCalculator.java + Driver.java)

Code Screenshot:



Output Screenshot:



Learning Outcome:

- Mastered upper-bounded type parameters using .
- Accessed common superclass methods (doubleValue()) directly on generic types.
- Enforced compile-time restrictions preventing invalid non-numerical arguments.
- Processed diverse wrapper types under a single algorithm.
- Deepened appreciation for Java's strong generic type system.

PRACTICAL - 13

Aim: Generic stack (For Advanced Learners) (GenericStack.java + Driver.java)

Code Screenshot:

Visual Studio Code - GenericStack.java

GenericStack.java

```java
import java.util.NoSuchElementException;

public class GenericStack<T> {
    private Object[] items;
    private int size;

    public GenericStack(int capacity) { items = new Object[Math.max(1, capacity)]; }
    public void push(T value) {
        if (size == items.length) items = java.util.Arrays.copyOf(items, items.length * 2);
        items[size++] = value;
    }
    @SuppressWarnings("unchecked")
    public T pop() {
        if (size == 0) throw new NoSuchElementException("Stack is empty");
        T value = (T) items[--size];
        items[size] = null;
        return value;
    }
    public boolean isEmpty() { return size == 0; }
}
```

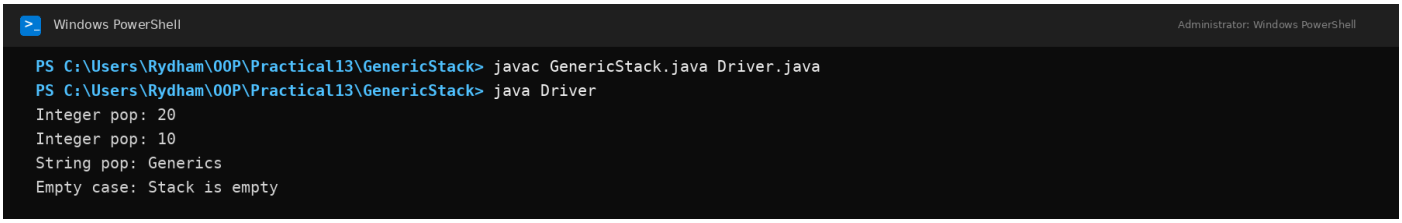
Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        GenericStack<Integer> numbers = new GenericStack<>(2);
        numbers.push(10);
        numbers.push(20);
        System.out.println("Integer pop: " + numbers.pop());
        System.out.println("Integer pop: " + numbers.pop());

        GenericStack<String> words = new GenericStack<>(2);
        words.push("Java");
        words.push("Generics");
        System.out.println("String pop: " + words.pop());
        try {
            words.pop();
            words.pop();
        } catch (java.util.NoSuchElementException e) {
            System.out.println("Empty case: " + e.getMessage());
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Implemented a reusable generic stack backed by an array.
- Reused the same stack class with Integer and String values.
- Expanded the backing array when additional capacity was required.
- Applied LIFO ordering through push() and pop() operations.
- Handled attempts to remove an item from an empty stack safely.